Checkable memory for agents:
an audit of what an agent-built memory store can honestly claim

Anshul Raj

Research paper · November 2026 · Project repository: https://github.com/Anshul48/trace

Abstract

When coding agents write a system's code, run its experiments and also write the reports on those experiments, how does anyone know which of the reported results are true? The reports can agree with the code and both still be wrong. We describe a small audit procedure for agent-built software and apply it to our own system, TRACE, a memory store for coding agents that answers a query by scoring only a small, routed shortlist of stored passages instead of the whole store. The procedure records whether each study had a plan fixed before it ran, ties every reported number to an evidence file, and has a separate analyst recompute selected findings. One result held: routing scored 0.79 percent of rows on every shard (partition) and, averaged over 4 shards, kept 0.9985 of the exact top ten results (scoring work only, not speed; no plan file). Several did not: search by meaning found fewer labelled sources than plain keyword search (26/50 vs 41/50); two refinements did not beat keyword search; and the live service was far slower than a standard index. When we planted 18 errors in a copy of this paper, the automated check caught 10 and missed 8: it catches wrong values and misses wrong meanings. The results concern one project; downstream coding performance and the general effectiveness of the procedure were not evaluated.

1 The problem: agents grading their own work

A scientist reading a result wants to know: what would make this number wrong, and did anyone check? That gets harder when software agents do the work. In our project, coding agents wrote the code, ran the experiments and wrote the reports. A generated report can agree with the generated code while both measure the wrong thing, so agreement between them is not evidence. Early on, a summary quoted a keyword-search score that belonged to a different experimental arm (see Corrections and limits); nothing looked wrong until someone traced the number to its file.

The system is TRACE, the memory layer of an agent system we are building. Its job is to let a coding agent find the right piece of a large body of project text (code, notes, reports) without reading all of it. The store studied here holds 812,545 passages of text, split into 4 separate partitions called shards, and was sealed, meaning no rows changed during the studies. Each passage is stored with an embedding, a list of numbers that encodes its meaning, so that passages with similar meaning lie close together. Searching by comparing embeddings is called dense search. TRACE answers a query in two steps: a routing step picks a shortlist of passages, and only that shortlist is compared exactly with the query. It then returns the wider text around each match, so the agent receives context and not a fragment.

The longer-term design goal for TRACE is a memory that reorganises itself as it is used. Nothing in this paper tests that goal. We ask a narrower question about the store as it exists today: what can we honestly say it does?

2 Related work

The procedure borrows from reforms of empirical practice: fixing the analysis plan before seeing the data (Nosek et al.) and the reproducibility checklists of machine-learning venues (Pineau et al.). What differs is the subject: results produced by coding agents, where code, experiment and report have the same kind of author. Benchmarks for agentic software engineering such as SWE-bench (Jimenez et al.) measure whether a task was solved, not whether an agent's own reported results can be trusted. On the retrieval side, our negative result agrees with the BEIR benchmark (Thakur et al.), in which keyword scoring (Robertson and Zaragoza) stayed a strong baseline against dense retrievers (Karpukhin et al.); our rank fusion is reciprocal rank fusion (Cormack et al.). Agent memory systems such as MemGPT (Packer et al.) and retrieval-augmented generation (Lewis et al.) motivate the store itself.

3 The audit procedure

The procedure checks files, not intentions, and has four parts.

Plan status. Each study is labelled by when its plan was fixed: frozen in advance (a plan file stating the question, the method, the measure and the rule for the verdict, fingerprinted with a hash before the run), historical (measured with no such plan file), or specified after looking at the data. A hash is a short fingerprint of a file that changes if the file changes. Ours are recorded locally, so they show a plan was not edited later only if that local record is itself untouched: the freezes are self-attested.

Receipts. Every result written in digits in the prose carries a receipt: the file it came from, the hash of that file, and the line or key where the value sits. A build step, which we call the gate, fails when a digit-form number it recognises (outside its

exemptions) has no receipt, when a pinned file has changed, or when the printed value differs from the receipted one. Numbers spelled out in words are invisible to the gate, so wording still needs review.

Separate recomputation. A second analyst recomputed selected results from raw files: the head-to-head verdicts, the rank-fusion and hierarchy figures, and the oracle-gap study. Results recomputed this way are marked OBSERVED. Results backed only by a fingerprinted summary file are marked REPORTED; the score-fusion count, for example, was taken from the worker model (an AI agent) that ran it. Only selected results were recomputed, and recomputation checks values, not meaning.

Wording limit. A fixed list restricts what the text may say about each result (for example, 'prove' is disallowed), and a table linking each claim to its evidence ships alongside the paper.

Passing sampled tests is not equivalence. Agent workflows commonly accept a rewritten program when it agrees with the original on a sample of test inputs. We ran a control, planned in advance, on our own acceptance tests: at the largest declared input range, the tests passed while the two programs differed, in 200 of 200 seeds. So we label such acceptances sample-tested only: agreement on the inputs tried, not sameness. This is what sampling arithmetic predicts, not a discovery about real test suites. The figure shows the whole flow with three examples from this project.

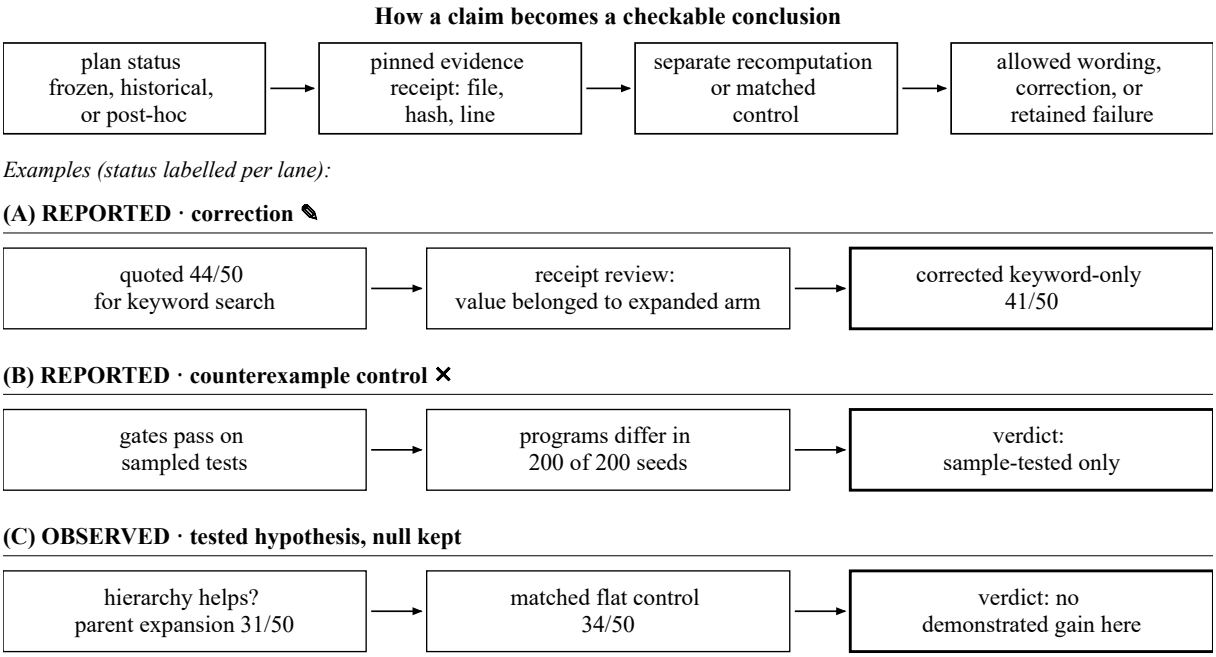


Figure 1: Claim flow: plan status, pinned evidence, separate recomputation or control, then allowed wording, correction, or retained failure. Lane A is a historical correction: keyword search quoted at 44/50 was the expanded-context arm, corrected to 41/50 for keyword search alone. Lane B is the control described above: gates passed while programs differed in 200 of 200 seeds, so such accept loops are sample-tested only. Lane C is a tested hypothesis kept as a null: parent expansion 31/50 against matched flat 34/50, no demonstrated gain here.

4 What the gate catches, and what it misses

To test the gate we seeded 18 faults into copies of an earlier snapshot of this paper (85 receipted numeric spans) and ran the gate on each copy, with 6/6 harmless control edits correctly passing. The gate caught 10 faults and missed 8. Every caught fault was mechanical: a digit-form number with no receipt, a changed pinned file, a span that differs from its receipted value, or a banned phrase. Every miss was semantic: a correct numeral attached to the wrong arm or denominator, a claim of prospective registration for a study whose table row says no plan file, a word-form number contradicting a receipted one, and a receipt id reused for a different quantity.

We chose the fault set ourselves, so it gives no error rate. It shows that the gate checks values and hashes, and that meaning needs a reader.

5 What survived the audit: results on the TRACE store

Unless stated otherwise, results use 50 test searches over the sealed store. A search scores a source hit when the exact passage the query was labelled with appears among the returned results. Source hit measures finding a known source; it does not measure answer quality.

Table 1: Claim table. OBSERVED: recomputed from raw files, some by the second analyst. REPORTED: a summary-backed figure we did not recompute. The preservation figure is a mean over 200 query-by-shard pairs (queries repeat across shards), and the scoring fraction holds on every shard.

Question and record level	Recorded result	What it allows us to say	Receipts
Does guided routing preserve the dense ranking while scoring fewer rows? OBSERVED (no plan file)	Exact top-ten overlap 0.9985; scored fraction 0.79 percent on every shard	Routing preserved this retriever's results at a small fraction of the scoring work. No inference about latency or usefulness	`e1a-recall-09985, e1a-rows-079`
Does dense search find the labelled source more often than keyword search? REPORTED, later reproduced as OBSERVED (frozen plan)	Keyword 41/50; dense 26/50	No. Keyword search found more labelled sources on these searches	`e2d-bm25-41-50, e2d-dense-26-50`
Does hierarchy help at matched (same-size) context? OBSERVED (frozen plan)	Expanded 31/50; flat 34/50	No demonstrated hierarchy gain; no equivalence claim	`e3a-e-hnsw-31-50, e3a-fk-hnsw-34-50`
Does combining keyword and dense scores beat keyword alone? OBSERVED (rank fusion, recomputed) and REPORTED (score-fusion count, taken from the worker) (frozen plan)	Rank fusion 38/50; score fusion 40/50; keyword 41/50	No gain over keyword in these fixed configurations; not a universal rejection of hybrids	`e3b-rrf-38-50, e3b-minmax-40-50, e3b-bm25-41-50`
Could an ideal chooser between keyword and dense do better than dense alone? OBSERVED (frozen plan)	Oracle gap +0.068 over dense in equal-style-average source-hit rate	The two methods find different sources on this constructed set; an oracle uses labels and is not a deployed router	`x1-gap`
Does fixed score fusion help inside the rows that routing admits? OBSERVED (stage 1 had no plan file; stages 2 to 6 were frozen before their runs, per our own freeze records)	The combination cleared its bar on 3 of the first 5 query sets and missed on 2	A mixed pattern (post-hoc tally, no joint test) with no established explanation	`cf2-dhit, cf3-dhit, cf4-dhit, cf5-m-dhit, cf5-h-dhit`

A hash-pinned summary is weaker than a raw-data recomputation; the separate analyst recomputed rank fusion but took score-fusion counts from the worker.

Routing does less scoring work and keeps the ranking. Guided routing scored 0.79 percent of rows on every shard and kept 0.9985 of the exact top ten, measured with 50 test searches keeping the top 10 per shard. This is a measure of scoring work, not of latency or usefulness, and the measured TRACE arm used routed dense search only, with no keyword component.

Search by meaning finds fewer labelled sources than keyword search. Keyword (word-match) search found the labelled source in 41/50 searches and dense search in 26/50. A later frozen-protocol re-collection of the same searches by an independent evaluator reproduced both counts: keyword 41 of 50, routed dense search 26 of 50.

Hierarchy and simple fusion did not help. Returning hierarchy-expanded parent context reached 31/50 against a flat arm of matched size at 34/50. Two ways of combining keyword and dense results, rank fusion and score fusion, reached 38/50 and 40/50 against keyword search alone at 41/50. These failures are kept at the same record level as the positive result.

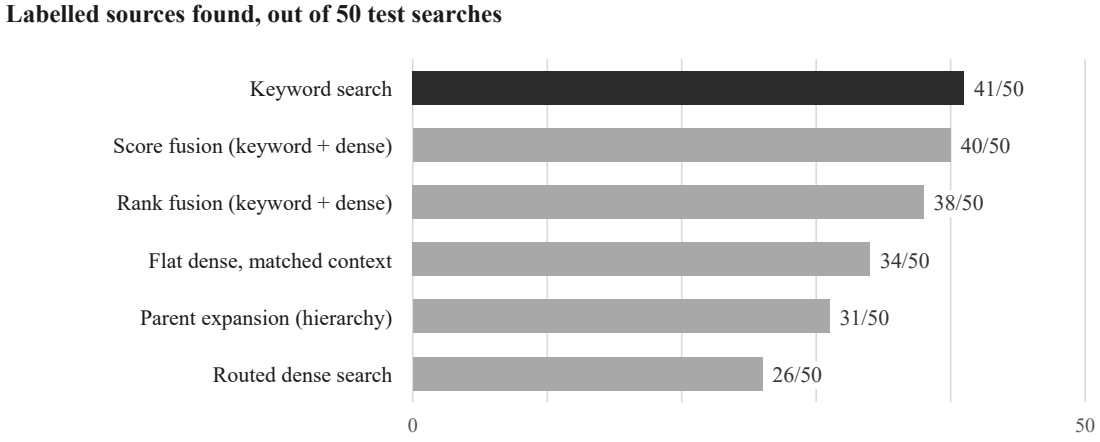


Figure 2: Labelled sources found, out of 50 test searches, by method. The values are those of the claim table: keyword 41/50, score fusion 40/50, rank fusion 38/50, flat context 34/50, parent expansion 31/50, routed dense 26/50. The bars come from three separate comparisons on the same searches, not one ranked experiment.

Keyword and dense search are complementary on a constructed set. On a set of model-written queries, an ideal chooser that always picks the better of keyword and dense search (it cheats by using the labels) gains +0.068 in source-hit rate over dense search alone, averaged over three query styles (question, keyword, description). So the two methods find different sources: room for a chooser, not a working one. The test used one embedding model (MiniLM-L6-v2) on one corpus, without routing. On the original fifty searches the same ideal chooser gains only +2/50 over keyword search (REPORTED).

Combining the two inside the route gave a mixed pattern. We then tested a fixed, equal-weight combination of keyword and dense scores on the rows that routing admits for a query (on average 13.4% of the store). Across the first five sets of model-written queries, each with its pass bar fixed before the run, the combination cleared the bar on 3 of 5 and missed on 2 (a tally made afterwards, with no joint test). Two follow-up tests did not explain it: the effect of who wrote the queries was +0.030 [-0.009,0.071], and the planned test of word overlap between query and passage gave +0.000 [-0.050,0.050]; both ranges include zero. Why the gains varied is unestablished; nothing here licenses a deployed hybrid.

Speed: a standard index is far faster at the search stage. In the same-store head-to-head, the fastest quality-matched (same answer quality) standard-index configuration recorded a median of 0.66 milliseconds (search only, 1 thread, ids only, shared virtual machine) against 1236 milliseconds for the live guided service, a ratio of 1884. The preregistered verdict reads that guided search is dominated by a standard index on this store. Embedding on both sides leaves a ratio near 86. This compares search-stage cost only: the service returns parent text, the standard index ids only. Waiting on shard-engine processes, which run in sequence, took 88.8 to 91.2 percent of guided query time in a profile (REPORTED); launch and scoring were not separated. We make no general speed claim.

Memory: the preregistered advantage criterion failed. In the memory-at-scale comparison, the ratio of TRACE's process memory to a standard graph index's was 0.1675 at small size, 0.1717 at middle size, and 0.1721 at a third, largest store size (one size is a worker figure we did not re-time). The ratio did not fall with scale, so it failed the preregistered memory-advantage criterion. This process memory omits the page cache, and we draw no memory claim.

6 Corrections and limits

One correction shows the procedure narrowing a claim. An earlier note quoted 44/50 for keyword search, which was the expanded-context arm; the keyword-only figure is 41/50. The error was found by reading receipts, not through the automated gate.

Queries were authored, not sampled from user traffic; one model family both wrote and filtered the first synthetic set, so author and judge effects were never separated, and no synthetic query was written by a person. A different-model stand-in transfer check on three small public sets did not reproduce the headline recall and scoring work (REPORTED, words only), so the scoring-work figure may not transfer. Source hit counts only the exact source passage. The sealed-store studies use one corpus and a single embedding model (the memory comparison used another store), and 50 test searches give coarse resolution.

7 Reproduction

An experiment bundle re-derives the small-experiment results with one command (reruns on the author's host reproduced the shipped verdicts, with three analyses byte-identical; other systems are untested). The sealed-store runs need the author's virtual

machine and index; for those, a receipt bundle holds the pinned result files and the checker confirms that each numeric span matches its file. It confirms values and hashes only.

8 What the audit tells us to build next

Each failure points at a specific change. None of the mechanisms below exists in the evaluated system; each is a hypothesis that must pass the same gate.

Speed. Most of the guided query time went to waiting on shard processes started in sequence. Hypothesis: keeping those workers alive between queries removes most of the search-stage gap. Test: the same head-to-head, with the same verdict rule.

Finding sources. Keyword search beat dense search, and an ideal chooser beat both on a constructed set. Hypothesis: a per-query chooser between the two recovers part of that gap. Test: a bar fixed in advance, on fresh queries written by people.

The gate. Every planted error of meaning passed. Hypothesis: receipts that also record which experimental arm and which denominator a number belongs to will catch the right-number-wrong-arm class. Test: the planted-error challenge again, with errors written by someone else.

A store that organises itself. The store studied here is static: fixed shards, fixed routing. Hypothesis: grouping passages that are retrieved together, and letting unused links fade, lowers scoring work further while preserving the ranking. Test: the same preservation measure that the one positive result here was held to.

9 What we take from this

For TRACE, the audit leaves one positive result standing and a plain list of things the current store does not do better than simple baselines. For agent-built software generally, a mechanical gate is cheap and catches mechanical errors, but it does not replace a reader: every error of meaning we planted passed it. Keeping the failures at the same record level as the successes is what made the one success credible to us.

10 References

B. Nosek et al. The preregistration revolution. PNAS, 2018.

J. Pineau et al. Improving reproducibility in machine learning research. JMLR, 2021.

C. E. Jimenez et al. SWE-bench: can language models resolve real-world GitHub issues? ICLR, 2024.

N. Thakur et al. BEIR: a heterogeneous benchmark for zero-shot evaluation of information retrieval models. NeurIPS, 2021.

S. Robertson, H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in IR, 2009.

V. Karpukhin et al. Dense passage retrieval for open-domain question answering. EMNLP, 2020.

G. Cormack et al. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. SIGIR, 2009.

C. Packer et al. MemGPT: towards LLMs as operating systems. arXiv preprint, 2023.

P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS, 2020.

AI-assistance disclosure

Built with AI coding agents (Muse Code, Claude, Codex) under human architectural direction. Digit-form numbers are checked at build time against hash-pinned result files (some of them summaries); findings stated in words are not receipted.

Appendix A Receipted evidence

One row per receipt id cited by the Writeup, in order of first appearance. Evidence paths under owner-hub are shown relative to owner-hub/ (full path in the manifest and in each cell tooltip). CURRENT sha256 is the evidence file hash at build time (2026-11-12T21:49:51Z).

#	id	displayed value	tag	evidence file	line or key	recorded sha256	current sha256	status
1	e1a-rows-079	0.79 *cited 3x*	OBSERVED	scratch/wave6/paper/evidence-vm/shard-fraction-all/per_shard.tsv	2	78fb3c4db2298d3e01b34dafc4ac55dec3f7e6a56b89d461638a3c9762ab9964	78fb3c4db2298d3e01b34dafc4ac55dec3f7e6a56b89d461638a3c9762ab9964	MATCH
2	num-shards-4	4 *cited 2x*	REPORTED	scratch/wave6/claude-sci/h2h/baselines-mirror/RESULTS.md	9	0d6f09952ac72d71960153562cd794f302ea176c0c9b1f9842683a9dafc24912	0d6f09952ac72d71960153562cd794f302ea176c0c9b1f9842683a9dafc24912	MATCH
3	e1a-recall-09985	0.9985 *cited 3x*	OBSERVED	scratch/wave6/paper/evidence-facts/FACTS-AND-RULES.md	23	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	MATCH
4	e2d-dense-26-50	26/50 *cited 4x*	REPORTED	scratch/wave6/claude-sci/hier/summary_hl.json	F10_hnsw.hits/F10_hnsw.n	c5c3fea0d7b9b6e3af6d757d3936b25695fcf1e0bdb3dbf0fe894dff96d81cb2	c5c3fea0d7b9b6e3af6d757d3936b25695fcf1e0bdb3dbf0fe894dff96d81cb2	MATCH
5	e2d-bm25-41-50	41/50 *cited 4x*	REPORTED	scratch/wave6/claude-sci/hier/RESULT.md	42	e577ef0f71d69ee6f687079bffce66441619f3ed633a677a1a2e21ecd9578e37	e577ef0f71d69ee6f687079bffce66441619f3ed633a677a1a2e21ecd9578e37	MATCH
6	fc-seeded-18	18 *cited 2x*	REPORTED	scratch/wave6/paper/evidence-fc/RESULT.md	Headline: Seeded faults: 18	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	MATCH
7	fc-caught-10	10 *cited 2x*	REPORTED	scratch/wave6/paper/evidence-fc/RESULT.md	Headline: Checker caught 10	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	MATCH
8	fc-missed-8	8 *cited 2x*	REPORTED	scratch/wave6/paper/evidence-fc/RESULT.md	Headline: missed 8	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	MATCH
9	e1a-units-812545	812,545	REPORTED	scratch/wave6/paper/evidence-facts/FACTS-AND-RULES.md	22	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	MATCH
10	e2fz-200of200-2e20	200 of 200 *cited 2x*	REPORTED	scratch/wave6/claude-sci/e2/run.log	7	0597f54106d245c68479cd7809cb5b313a6cba68e997375a32621773136383e6	0597f54106d245c68479cd7809cb5b313a6cba68e997375a32621773136383e6	MATCH
11	l8-bm25-44-50	44/50 *cited 2x*	REPORTED	scratch/wave6/paper/positions/corrections.md	2	466a524fc5255db18fc02264a39f6895b4bf959715c005bec925956569f0a8a4	466a524fc5255db18fc02264a39f6895b4bf959715c005bec925956569f0a8a4	MATCH
12	l8-receipt-41-50	41/50 *cited 2x*	REPORTED	scratch/wave6/paper/positions/corrections.md	2	466a524fc5255db18fc02264a39f6895b4bf959715c005bec925956569f0a8a4	466a524fc5255db18fc02264a39f6895b4bf959715c005bec925956569f0a8a4	MATCH
13	e3a-e-hnsw-31-50	31/50 *cited 4x*	OBSERVED	scratch/wave6/claude-sci/hier/RESULT.md	40	e577ef0f71d69ee6f687079bffce66441619f3ed633a677a1a2e21ecd9578e37	e577ef0f71d69ee6f687079bffce66441619f3ed633a677a1a2e21ecd9578e37	MATCH
14	e3a-fk-hnsw-34-50	34/50 *cited 4x*	OBSERVED	scratch/wave6/claude-sci/hier/RESULT.md	40	e577ef0f71d69ee6f687079bffce66441619f3ed633a677a1a2e21ecd9578e37	e577ef0f71d69ee6f687079bffce66441619f3ed633a677a1a2e21ecd9578e37	MATCH
15	fc-snapshot-85	85	REPORTED	scratch/wave6/paper/evidence-fc/RESULT.md	11	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	MATCH
16	fc-controls-6	6/6	REPORTED	scratch/wave6/paper/evidence-fc/RESULT.md	Controls 6/6 pass	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	bcee2ab877f478ffb32cfd7eaf89ab336c70035cefd05271ac27594becaf7477	MATCH
17	e1a-n50	50 *cited 6x*	REPORTED	scratch/wave6/paper/evidence-facts/FACTS-AND-RULES.md	23	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	MATCH
18	mfid-n200	200	REPORTED	scratch/wave6/claude-sci/h2h/baselines-mirror/RESULTS.md	23	0d6f09952ac72d71960153562cd794f302ea176c0c9b1f9842683a9dafc24912	0d6f09952ac72d71960153562cd794f302ea176c0c9b1f9842683a9dafc24912	MATCH
19	e3b-rrf-38-50	38/50 *cited 3x*	OBSERVED	scratch/wave6/claude-sci/hier/HIER-FINDING.md	11	d3af34951771c1789833b4dbeed3996bfccf238a3c870e7e689651c08c6805e4	d3af34951771c1789833b4dbeed3996bfccf238a3c870e7e689651c08c6805e4	MATCH
20	e3b-minmax-40-50	40/50 *cited 3x*	REPORTED	scratch/wave6/claude-sci/hier/HIER-FINDING.md	11	d3af34951771c1789833b4dbeed3996bfccf238a3c870e7e689651c08c6805e4	d3af34951771c1789833b4dbeed3996bfccf238a3c870e7e689651c08c6805e4	MATCH
21	e3b-bm25-41-50	41/50 *cited 2x*	OBSERVED	scratch/wave6/claude-sci/hier/HIER-FINDING.md	11	d3af34951771c1789833b4dbeed3996bfccf238a3c870e7e689651c08c6805e4	d3af34951771c1789833b4dbeed3996bfccf238a3c870e7e689651c08c6805e4	MATCH
22	x1-gap	+0.068 *cited 2x*	OBSERVED	scratch/wave6/claude-sci/x1/vm/RESULT.md	19	492c12c17d22a0d013abef7033703f3d95903f51746a4ccb3e3efa29dd8aad6d	492c12c17d22a0d013abef7033703f3d95903f51746a4ccb3e3efa29dd8aad6d	MATCH

#	id	displayed value	tag	evidence file	line or key	recorded sha256	current sha256	status
23	dt-five-cleared-3	3 *cited 2x*	REPORTED	scratch/wave6/paper/evidence-derived/tally-five-sets.txt	12	bec3c4376e98b750ec12aefbff3583a1e60cf29934a8efc6c0b176cb9eed6a99	bec3c4376e98b750ec12aefbff3583a1e60cf29934a8efc6c0b176cb9eed6a99	MATCH
24	num-five-sets-5	5 *cited 2x*	REPORTED	scratch/wave6/paper/evidence-cfuse/c-fuse-5/OWN-CHECK.md	6	5d2812c93b2b1a43ba4c681e3e35e1d61f6d35dc63d66fee937bda507a925bce	5d2812c93b2b1a43ba4c681e3e35e1d61f6d35dc63d66fee937bda507a925bce	MATCH
25	dt-five-missed-2	2 *cited 2x*	REPORTED	scratch/wave6/paper/evidence-derived/tally-five-sets.txt	12	bec3c4376e98b750ec12aefbff3583a1e60cf29934a8efc6c0b176cb9eed6a99	bec3c4376e98b750ec12aefbff3583a1e60cf29934a8efc6c0b176cb9eed6a99	MATCH
26	e1a-k10	10	REPORTED	scratch/wave6/paper/evidence-facts/FACTS-AND-RULES.md	23	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	6a4434c26ab20e7e6ffa2876b5e02fcec3c59e498342a731b74f4a57de7ef249	MATCH
27	sealed50-k-41	41	OBSERVED	scratch/wave2/sealed-eval-stage2b-v1/metrics-report.txt	17	092da4856d110d02743dc044404112c1d8189472a37d6a928fd41c8b8d917543	092da4856d110d02743dc044404112c1d8189472a37d6a928fd41c8b8d917543	MATCH
28	sealed50-g-26	26	OBSERVED	scratch/wave2/sealed-eval-stage2b-v1/metrics-report.txt	18	092da4856d110d02743dc044404112c1d8189472a37d6a928fd41c8b8d917543	092da4856d110d02743dc044404112c1d8189472a37d6a928fd41c8b8d917543	MATCH
29	r21-x0-styles	three	REPORTED	scratch/wave6/paper/evidence-derived/runner21/query-split.json	style_count_word	a6bcb3c04b568897a0b757114d1445255545e88eaf21bccd7d7e2eda6adce093	a6bcb3c04b568897a0b757114d1445255545e88eaf21bccd7d7e2eda6adce093	MATCH
30	old50-oracle-gain	+2/50	REPORTED	scratch/wave6/paper/positions/corrections.md	3	466a524fc5255db18fc02264a39f6895b4bf959715c005bec925956569f0a8a4	466a524fc5255db18fc02264a39f6895b4bf959715c005bec925956569f0a8a4	MATCH
31	cf2-cand-pct	13.4%	OBSERVED	scratch/wave6/paper/evidence-cfuse/c-fuse-2.result.md	60	571cd513aedd44609b4073bf67ba6d32d29a66c12a6c1194966ae26ef2b251f4	571cd513aedd44609b4073bf67ba6d32d29a66c12a6c1194966ae26ef2b251f4	MATCH
32	cf5-paired	+0.030 [-0.009,0.071]	VERIFIED	scratch/wave6/paper/evidence-cfuse/c-fuse-5/RESULT.md	93	a559f63031c12262e9898eeb1c614d332bdb6ea335cc120037c157c351fff3cb	a559f63031c12262e9898eeb1c614d332bdb6ea335cc120037c157c351fff3cb	MATCH
33	cf6-d	+0.000 [-0.050,0.050]	VERIFIED	scratch/wave6/paper/evidence-cfuse/c-fuse-6/RESULT.md	5	dcf4f6465587fbb817eec60c680529d35deafa5b80453014 41ae4967090ac113	dcf4f6465587fbb817eec60c680529d35deafa5b80453014 41ae4967090ac113	MATCH
34	e2a-p50-066	0.66	OBSERVED	scratch/wave6/claude-sci/h2h/H2H-FINDING.md	7	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	MATCH
35	num-thread-1	1	REPORTED	scratch/wave6/claude-sci/h2h/H2H-FINDING.md	7	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	MATCH
36	e2b-guided-1236	1236	OBSERVED	scratch/wave6/claude-sci/h2h/H2H-FINDING.md	7	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	MATCH
37	e2b-r-1884	1884	OBSERVED	scratch/wave6/claude-sci/h2h/H2H-FINDING.md	7	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	MATCH
38	e2b-rend2end-86	86	OBSERVED	scratch/wave6/claude-sci/h2h/H2H-FINDING.md	7	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	ec162f0d51819ce311d11e553b28e23bca9f6d5f363dc614914a78bc2e33c9cd	MATCH
39	lat-engine-888	88.8	REPORTED	scratch/wave6/paper/evidence-vm/latency-profile-r2/vm-trace-latency-profile-v1-r2.result.md	13	a6d9344841a093f9f03c812004564770daf39800a4aeaaf68c945efdac09d14e	a6d9344841a093f9f03c812004564770daf39800a4aeaaf68c945efdac09d14e	MATCH
40	lat-engine-912	91.2	REPORTED	scratch/wave6/paper/evidence-vm/latency-profile-r2/vm-trace-latency-profile-v1-r2.result.md	13	a6d9344841a093f9f03c812004564770daf39800a4aeaaf68c945efdac09d14e	a6d9344841a093f9f03c812004564770daf39800a4aeaaf68c945efdac09d14e	MATCH
41	d3-ratio-0p8	0.1675	OBSERVED	scratch/wave6/paper/d3/p0/FINDING-D3-0p8M.md	9	73a4e594ea5946a0cd76bf294f0e23e2ae7634e1a53f727f88e448503622a2a7	73a4e594ea5946a0cd76bf294f0e23e2ae7634e1a53f727f88e448503622a2a7	MATCH
42	d3-ratio-3p2	0.1717	OBSERVED	scratch/wave6/paper/d3/p1/FINDING-D3-3p2M.md	Ratios per prereg	ee58eae16ef10a85e0ec01a034e0278638e3fc3e2854bc1ff5da2dd74318c9ad	ee58eae16ef10a85e0ec01a034e0278638e3fc3e2854bc1ff5da2dd74318c9ad	MATCH
43	d3-ratio-5p0	0.1721	OBSERVED	scratch/wave6/paper/evidence-vm/d3-5m-r5/d3-5p0m-v1-r5.result.md	173	a8d51d768757e87c86a2a4b18fe32f68d4cf3fd1979f391542901d59a8ef0632	a8d51d768757e87c86a2a4b18fe32f68d4cf3fd1979f391542901d59a8ef0632	MATCH
44	r21-identical-analyses	three	REPORTED	scratch/wave6/paper/evidence-derived/runner21/reproduction-tally.json	identical_analyses_word	a7d49239d4efe01758581e6549f860edaf9c642816a3f0842a6137ad3c13f032	a7d49239d4efe01758581e6549f860edaf9c642816a3f0842a6137ad3c13f032	MATCH

Provenance. Built 2026-11-12T21:49:51Z | writeup html sha256: d510d5a1d21c40d99243441c28baa1e451260c16186aaf421d5341f60ceb244d | receipts json sha256: 46d2da1150430a0272984a34628786091a55a00d92e4fd2e4129631d0c847089 | checker exit=0: OK: scratch/wave6/paper-pro-20261112/arxiv/writeup-arxiv.html (81 receipted spans, 272 text segments clean) | 44 cited receipts: 44 MATCH, 0 MISMATCH, 0 MISSING, 0 UNKNOWN; 0 known-lost uncited (MISSING)